



Imersão GenAI para Desenvolvedores

Módulo 1

Preâmbulo: Neste módulo, você configurará seu ambiente de desenvolvimento com VSCode e IA generativa e reforçará seus conhecimentos de Python em exercícios práticos

Versão: 1.1

Sumário

I	Instruções Importantes	2
II	Antes de começar	4
III	Exercício 1: Variáveis	6
IV	Exercício 2: Valores Booleanos	7
V	Exercício 3: Listas	8
VI	Exercício 4: Dicionários	9
VII	Exercício 5: Argumentos de linha de comando	10
VIII	Exercício 6: Minha primeira função	11
IX	Exercício 7: Ambiente Virtual	12
X	Exercício 8: API Requests	14
XI	Exercício 9: Assistente de AI	15
XII	Entrega e Avaliação entre Pares	16

Capítulo I

Instruções Importantes

- Use esta página como sua **única referência**. Não escute rumores ou especulações sobre como preparar sua solução.
- **Leia todo o documento com atenção antes de iniciar**. Dicas valiosas podem estar ao final de cada exercício.
- **Leia os Exemplos com Atenção**. Eles podem exigir coisas que **não estão especificadas no enunciado**.
- Para ser avaliada, sua solução deve estar no seu **repositório GIT**.
- Suas soluções serão avaliadas pelos seus colegas de curso.
- Você **não deve deixar outros arquivos** no seu diretório além daqueles explicitamente especificados nas instruções do exercício.
- Quando precisar obter uma saída precisa nos seus programas, **você não deverá** exibir uma saída pré-calculada em vez de realizar o exercício corretamente.
- Tem uma dúvida? Pergunte para o seu colega à direita. Se isso não funcionar, tente o colega à esquerda.
- Como **materiais de referência**, você pode utilizar o **seus pares**, o **manual das ferramentas**, e **internet** em geral.



Aqui, o aprendizado acontece por meio da experimentação, da tentativa e erro e da colaboração. Se estiver com dificuldades, peça ajuda a seus colegas e, juntos, busquem fontes confiáveis. Evite usar IA para gerar todo o código ou resolver exercícios por você. Aprender a enfrentar desafios sem atalhos fortalecerá suas habilidades de maneira duradoura. Utilize o assistente de IA para auxiliar no seu desenvolvimento, mas não entregue um código que você não compreende.

- Aqui e adiante, usamos o **Python 3.10** como referência. Você pode checar a versão com o comando `python3 -V`
- Para esta imersão, utilizaremos o **VSCode** como editor padrão.
- A menos que especificado de forma diferente em determinados exercícios, todos os componentes da **biblioteca padrão do python** podem ser utilizados.
- O uso das bibliotecas e ferramentas externas `flake8`, `ruff`, `mypy`, `pytest`, `requests`, `httpx`.
- Você também pode utilizar outras bibliotecas que julgar necessário, desde que ela não faça todo o trabalho para você.
- Nomes descritivos para variáveis e funções, modularização do código, tratamento adequado de erros e comentários quando necessário podem ajudar durante a avaliação.

E que a Força esteja com você!

Capítulo II

Antes de começar

Seus projetos serão sempre entregues utilizando repositórios **Git**. Para que você possa acessar estes repositórios, criados automaticamente pela Intranet da 42, você deve criar um par de chaves SSH, e inserir a chave pública na página dedicada às chaves SSH em seu perfil na intranet. Para isto, siga as instruções abaixo:

1. Abra o terminal e digite: `ssh-keygen -t ed25519`
2. Pressione a tecla ENTER três vezes (para utilizar as configurações padrão), até que você perceba algo semelhante à isto:

```
+--[ED25519 256]--+
|      ..+oo+++o |
|      +...++o. |
|      . o. +.. o |
|      o  + o   |
|      So+ *    |
|      .  o..B . |
| .oo oEo .  =  |
| .=o+...+   o  |
| .oo*o.o..o   |
+-----[SHA256]-----+
```

3. Agora, execute o seguinte comando para visualizar sua chave pública:

```
?> cat ~/.ssh/id_ed25519.pub
```

4. Pegue a chave SSH exibida e copie. Você deve colocá-la no seu perfil da Intra.
 - Na página do seu perfil na Intra, selecione o seu login no canto superior direito > Settings > SSH Key. Ou, acesse diretamente por este [link](#).
 - Clique em New SSH Key, cole a chave SSH no campo e envie.
5. Por fim, configure seu **git** para utilizar, globalmente, seu nome e email para registrar cada commit:

```
?> git config --global user.name "<SEU NOME>"
?> git config --global user.email "<SEU EMAIL>"
```



Utilizamos `'?>'` como um marcador visual para comandos executáveis, sempre que houver um exemplo de uso do terminal. Esse prefixo serve apenas como indicação - **ele não deve ser digitado** junto com o comando.

Capítulo III

	Exercício : 01
Variáveis	
Pasta de entrega : <code>ex01/</code>	
Arquivos para entregar : <code>name.py</code>	
Funções ou bibliotecas autorizadas : Biblioteca padrão	

1. Crie um script chamado `name.py`
2. Defina uma variável chamada `first_name` e a inicialize com o seu primeiro nome.
3. Defina uma variável chamada `last_name` e a inicialize com o seu último nome.
4. Mostre ambas variáveis na saída padrão do terminal, seguida por uma quebra de linha.

Exemplo de Saída Esperada:

```
?> python3 name.py
Ford Prefect
?>
```

Capítulo IV

	Exercício : 02
Valores Booleanos	
Pasta de entrega : <i>ex02/</i>	
Arquivos para entregar : <code>boolean.py</code>	
Funções ou bibliotecas autorizadas : Biblioteca padrão	

Utilizando apenas duas variáveis, `a` e `b`, alcance a saída esperada:

```
?> python3 boolean.py
É igual? False
É maior? True
É menor? False
?>
```



Neste exercício você deve utilizar operadores de comparação.

Capítulo V

	Exercício : 03
Listas	
Pasta de entrega : <i>ex03/</i>	
Arquivos para entregar : lists.py	
Funções ou bibliotecas autorizadas : Biblioteca padrão	

1. Escreva um programa que imprime o conteúdo de uma lista `my_list`, um item por linha:

2. Exemplo do arquivo `lists.py`:

```
my_list = ['item1', 'item2', 'item3', 'item4']  
  
# seu código aqui.
```

3. Exemplo de saída esperada:

```
?> python3 lists.py  
item1  
item2  
item3  
item4  
?>
```

Capítulo VI

	Exercício : 04
Dicionários	
Pasta de entrega : <i>ex04/</i>	
Arquivos para entregar : dicts.py	
Funções ou bibliotecas autorizadas : Biblioteca padrão	

1. Escreva um programa que imprime o conteúdo de um dicionário `my_dict`, um item por linha, separando chave e valor por um espaço:
2. Exemplo do arquivo `dicts.py`:

```
my_dict = {  
    'item1': 1  
    'item2': 2  
    'item3': 3  
    'item4': 4  
}  
  
# seu código aqui.
```

3. Exemplo de saída esperada:

```
?> python3 dicts.py  
item1 1  
item2 2  
item3 3  
item4 4  
?>
```

Capítulo VII

	Exercício : 05
Argumentos de linha de comando	
Pasta de entrega : <i>ex05/</i>	
Arquivos para entregar : <code>arguments.py</code>	
Funções ou bibliotecas autorizadas : Biblioteca padrão	

1. Crie um programa que recebe uma **string** como argumento de linha de comando, e o imprima na tela.
2. Exemplo de execução esperada:

```
?> python3 arguments.py "42"  
42
```



`sys.argv`



Lembre-se de tratar o caso em que nenhum argumento é fornecido.

Capítulo VIII

	Exercício : 06
Minha primeira função	
Pasta de entrega : <code>ex06/</code>	
Arquivos para entregar : <code>first_function.py</code>	
Funções ou bibliotecas autorizadas : Biblioteca padrão	

1. Crie uma função no arquivo `first_function.py`, que recebe uma `string` e imprime uma saudação, com o seguinte protótipo:

```
def greeting(name: str) -> None:
    """
    receives a name and prints a greeting
    """
    # seu código
```

1. Sua função deverá ser chamada pelo seu programa, que passará a ela o primeiro argumento recebido na linha de comando:

```
?> python3 first_function.py 42
Hello 42
```



Você pode tratar o caso de não haver um argumento de linha de comando da forma que achar mais adequado, como por exemplo exigir uma mensagem de erro ou usar um valor padrão.

Capítulo IX

	Exercício : 07
Ambiente Virtual	
Pasta de entrega : <i>ex07/</i>	
Arquivos para entregar : <code>greetings_from_the_farm.py</code>	
Funções ou bibliotecas autorizadas : Biblioteca padrão, <code>cowsay</code>	

Quando trabalhamos com Python, frequentemente precisamos instalar pacotes adicionais usando um gerenciador de pacotes (utilizaremos o `_pip_` neste curso). No entanto, instalar pacotes globalmente pode causar conflitos entre projetos, pois diferentes aplicações podem exigir versões diferentes das mesmas bibliotecas.

1. Crie, e ative, um ambiente virtual no diretório base de seu usuário.

```
# Guia rápido:
python3 -m venv virtual_env      # Cria o ambiente
source virtual_env/bin/activate  # Ativa (Linux)
virtual_env\Scripts\activate.bat # Ativa (Windows)
```

2. No ambiente virtual ativado, instale a biblioteca externa '`cowsay`':

```
(virtual_env) ?> pip install cowsay
```

3. Altere sua função `greeting` para, ao invés de utilizar '`print`', utilizar um dos métodos da biblioteca `cowsay`:

```
?> python3 greetings_from_the_farm.py Bob
-----
| Hello Bob |
=====
      \      ^__^
       \      (oo)\_______
          (__)\       )\/\
             ||----w |
              ||     ||
```



Uma vez instalada, lembre-se de importar a biblioteca em seu módulo `'greetings_from_the_farm.py'`.



Você pode desativar seu ambiente virtual com o comando `'deactivate'`. Você deve ser capaz de explicar como ativar e desativar o ambiente virtual durante a avaliação, além dos motivos pelos quais ele deve ser utilizado.

Capítulo X

	Exercício : 08
API Requests	
Pasta de entrega : <i>ex08/</i>	
Arquivos para entregar : weather.py	
Funções ou bibliotecas autorizadas : Biblioteca padrão, requests, httpx	

1. Crie um programa chamado **weather.py** que recebe como argumento de linha de comando uma cidade e responde com a temperatura atual.
2. Seu programa deverá fazer um request para o endpoint abaixo, de forma a obter a latitude e longitude da cidade desejada.

```
f"https://geocoding-api.open-meteo.com/v1/search?name={city}&count=1"
```

3. Seu programa deverá fazer um request para o endpoint abaixo, para obter a temperatura local.

```
f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=temperature_2m"
```

4. Exemplo de saída esperada:

```
?> python3 weather.py "Glasgow"
Current temperature in Glasgow, United Kingdom is 17.6 °C
?>
```



Você pode obter mais informações sobre esta API [neste link](#).

Capítulo XI

Bônus

	Exercício : 09
	Assistente de AI
	Pasta de entrega : <code>ex09/</code>
	Arquivos para entregar : <code>README.md</code>
	Funções ou bibliotecas autorizadas : Biblioteca padrão



O exercício Bônus será avaliado somente se todos os exercícios anteriores estiverem sido realizados corretamente.

1. Escolha uma ferramenta de IA para desenvolvimento. Sugerimos o `continue.dev`, mas fique a vontade para utilizar um de sua preferência (ex: GitHub Copilot, Windsurf, etc.)
2. Siga o processo de instalação e configuração da ferramenta escolhida no VSCode.
3. utilize o agente de IA para criar um arquivo `README.md` que contenha uma descrição de cada um dos exercícios criados neste módulo.



Não é necessário assinar um plano para utilizar um assistente de IA. Busque um que possua um pacote 'free-tier'.

Capítulo XII

Entrega e Avaliação entre Pares

- Entregue seu projeto em seu repositório *Git* disponível na página do projeto na intranet.
- Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Não hesite em verificar os nomes de seus arquivos e pastas para garantir que estejam corretos.
- No horário da avaliação, o avaliador se dirigirá à estação de trabalho do aluno a ser avaliado para realizar os testes. Um clone do repositório deverá ser realizado em uma nova pasta, e estes são os arquivos que serão avaliados.